want to put a limit on how long you wait. This might be to allow you to send some form of "I'm still alive" message either to an interactive user or another process or indeed to allow you to abort the wait if the user has given up waiting and pressed Cancel.

There are two sorts of timeouts you may wish to specify: a *duration-based* timeout, where you wait for a specific amount of time (for example, 30 milliseconds), or an *absolute* timeout, where you wait until a specific point in time (for example, 17:30:15.045987023 UTC on November 30, 2011). Most of the waiting functions provide variants that handle both forms of timeouts. The variants that handle the duration-based timeouts have a `_for` suffix, and those that handle the absolute timeouts have a `_until` suffix.

So, for example, `std::condition_variable` has two overloads of the `wait_for()` member function and two overloads of the `wait_until()` member function that correspond to the two overloads of `wait()`—one overload that just waits until signaled, or the timeout expires, or a spurious wakeup occurs, and another that will check the supplied predicate when woken and will return only when the supplied predicate is `true` (and the condition variable has been signaled) or the timeout expires.

Before we look at the details of the functions that use the timeouts, let's examine the way that times are specified in C++, starting with clocks.

4.3.1. Clocks

As far as the C++ Standard Library is concerned, a clock is a source of time information. In particular, a clock is a class that provides four distinct pieces of information:

- The time *now*
- The type of the value used to represent the times obtained from the clock
- The tick period of the clock
- Whether or not the clock ticks at a uniform rate and is thus considered to be a *steady* clock

The current time of a clock can be obtained by calling the static member function `now()` for that clock class; for example, `std::chrono::system_clock::now()` will return the current time of the system clock. The type of the time points for a particular clock is specified by the `time_point` member `typedef`, so the return type of `some_clock::now()` is `some_clock::time_point`.

The tick period of the clock is specified as a fractional number of seconds, which is given by the `period` member `typedef` of the clock—a clock that ticks 25 times per second thus has a `period` of `std::ratio<1,25>`, whereas a clock that ticks every 2.5 seconds has a `period` of `std::ratio<5,2>`. If the tick period of a clock can't be known until runtime, or it may vary during a given run of the application, the `period` may be specified as the average tick period, smallest possible tick period, or some other value that the library writer deems appropriate. There's no guarantee that the observed tick period in a given run of the program matches the specified `period` for that clock.